

EdD-Pry2-ISem26

May 4, 2026

1 Proyecto II

1.1 Simulador de Gestión de Memoria Paginada usando Árboles AVL y Splay Trees

1.1.1 Contexto general

Se debe desarrollar en **C++** un simulador de gestión de memoria basado en **paginación**, donde múltiples procesos solicitan, liberan y referencian memoria. El sistema debe administrar las páginas físicas disponibles, las páginas asignadas a cada proceso y mecanismos de caché por proceso y caché global.

El proyecto debe utilizar obligatoriamente:

- **Árboles AVL.**
 - **Splay Trees.**
 - Listas, colas, pilas o estructuras auxiliares cuando sean necesarias.
 - Archivos de texto o entrada interactiva para leer operaciones.
-

1.1.2 Objetivo general

Diseñar e implementar un sistema de simulación de memoria paginada que permita administrar procesos, páginas físicas, referencias a memoria y cachés, utilizando árboles balanceados y autoajustables como estructuras centrales de búsqueda, inserción, eliminación y actualización.

1.1.3 Parámetros iniciales del sistema

Al iniciar el programa, el usuario o archivo de configuración debe indicar:

```
MEMORIA_TOTAL  
TAMANO_PAGINA  
TAMANO_CACHE_GLOBAL
```

Por ejemplo:

```
MEMORIA_TOTAL 65536  
TAMANO_PAGINA 1024  
TAMANO_CACHE_GLOBAL 10
```

Esto implica:

Cantidad total de páginas = $\text{MEMORIA_TOTAL} / \text{TAMANO_PAGINA}$

Ejemplo:

$65536 / 1024 = 64$ páginas físicas

1.1.4 Modelo de memoria

Cada proceso posee:

idProceso
memoriaSolicitada
paginasAsignadas
cacheLocal
tablaDePaginas

Cada página debe almacenar, como mínimo:

numeroPaginaVirtual
numeroPaginaFisica
idProceso
bitValida
bitModificada
contadorReferencias
ultimaReferencia

1.1.5 Uso obligatorio de estructuras

Árbol AVL El árbol AVL debe utilizarse para administrar páginas físicas libres u ocupadas, garantizando búsquedas eficientes.

Uso sugerido:

AVL de páginas libres
Clave: número de página física
Valor: información de la página

Operaciones mínimas del AVL:

insertar()
eliminar()
buscar()
rotarIzquierda()
rotarDerecha()
balancear()
inOrden()
altura()

Splay Tree El Splay Tree debe utilizarse para modelar accesos frecuentes a memoria, ya que cada referencia debe acercar la página consultada a la raíz.

Uso sugerido:

Splay Tree por proceso

Clave: número de página virtual

Valor: página asignada al proceso

También puede usarse un Splay Tree global para mantener páginas recientemente referenciadas.

Operaciones mínimas:

```
insertar()
eliminar()
buscar()
splay()
rotacionZig()
rotacionZigZig()
rotacionZigZag()
recorrido()
```

1.1.6 Operaciones del sistema

El programa debe aceptar operaciones por consola o desde archivo.

Crear proceso

```
NEW_PROCESS idProceso memoriaInicial
```

Ejemplo:

```
NEW_PROCESS P1 4096
```

Si el tamaño de página es 1024, entonces P1 requiere:

$4096 / 1024 = 4$ páginas

El sistema debe:

1. Verificar si existen páginas libres.
 2. Asignar páginas físicas.
 3. Crear la tabla de páginas del proceso.
 4. Crear su caché local.
 5. Registrar el proceso en la tabla global de procesos.
-

Finalizar proceso

```
END_PROCESS idProceso
```

Ejemplo:

END_PROCESS P1

El sistema debe:

1. Liberar todas las páginas físicas del proceso.
 2. Eliminar su tabla de páginas.
 3. Eliminar su caché local.
 4. Eliminar sus entradas del caché global.
 5. Retornar las páginas al AVL de páginas libres.
-

Referenciar una dirección de memoria

ACCESS idProceso direccionMemoria

Ejemplo:

ACCESS P1 2500

El sistema debe calcular:

$\text{paginaVirtual} = \text{direccionMemoria} / \text{TAMANO_PAGINA}$
 $\text{desplazamiento} = \text{direccionMemoria} \% \text{TAMANO_PAGINA}$

Luego debe:

1. Verificar si el proceso existe.
 2. Verificar si la página virtual existe.
 3. Buscar primero en el caché local.
 4. Si no está, buscar en el caché global.
 5. Si tampoco está, buscar en el Splay Tree del proceso.
 6. Actualizar métricas de hit/miss.
 7. Hacer splay sobre la página consultada.
 8. Actualizar la política de reemplazo si el caché está lleno.
-

Solicitar más memoria

ALLOCATE idProceso memoriaAdicional

Ejemplo:

ALLOCATE P1 2048

El sistema debe:

1. Calcular cuántas páginas adicionales se requieren.
 2. Verificar disponibilidad.
 3. Asignar nuevas páginas físicas.
 4. Insertarlas en el Splay Tree del proceso.
 5. Actualizar el tamaño máximo del caché local.
-

Liberar memoria de un proceso

`FREE idProceso direccionInicial cantidadBytes`

Ejemplo:

`FREE P1 2048 2048`

El sistema debe:

1. Calcular el rango de páginas virtuales afectadas.
 2. Eliminar esas páginas del Splay Tree del proceso.
 3. Liberar las páginas físicas correspondientes.
 4. Actualizar el AVL de páginas libres.
 5. Eliminar esas páginas del caché local y global.
 6. Ajustar el tamaño máximo permitido del caché local.
-

1.1.7 Caché local y caché global

Caché local por proceso Cada proceso debe tener un caché limitado a:

20% de las páginas asignadas al proceso

Ejemplo:

Proceso con 10 páginas asignadas

Cache local máximo = 2 páginas

Si el resultado no es entero, puede usarse:

`max(1, floor(paginasAsignadas * 0.20))`

El estudiante debe documentar su decisión.

Caché global Debe existir un caché global compartido entre todos los procesos.

Debe almacenar páginas recientemente referenciadas, por ejemplo:

`idProceso
paginaVirtual
paginaFisica
ultimaReferencia`

El tamaño del caché global será un parámetro del sistema.

Política de reemplazo El proyecto debe implementar al menos una política de reemplazo:

- LRU (Least Recently Used)

Se recomienda LRU para cachés.

1.1.8 Archivos de entrada

El sistema debe poder leer operaciones desde un archivo de texto.

Ejemplo:

```
MEMORIA_TOTAL 65536
TAMANO_PAGINA 1024
TAMANO_CACHE_GLOBAL 8
```

```
NEW_PROCESS P1 4096
NEW_PROCESS P2 8192
ACCESS P1 100
ACCESS P1 2500
ACCESS P2 7000
ALLOCATE P1 2048
FREE P2 2048 2048
END_PROCESS P1
END_PROCESS P2
```

1.1.9 Salidas esperadas

El programa debe mostrar información como:

```
Operación ejecutada
Estado de la memoria
Páginas libres
Páginas ocupadas
Estado de cada proceso
Contenido del caché local
Contenido del caché global
Hits y misses
Errores detectados
```

Ejemplo:

```
ACCESS P1 2500
Página virtual: 2
Desplazamiento: 452
Resultado: MISS en cache local, HIT en cache global
Página física: 12
```

1.1.10 Métricas obligatorias

El sistema debe calcular:

```
Total de accesos
Hits en caché local
Misses en caché local
```

Hits en caché global
Misses en caché global
Cantidad de páginas asignadas
Cantidad de páginas libres
Porcentaje de utilización de memoria

1.1.11 Manejo de errores

El sistema debe detectar y reportar:

Proceso inexistente
Proceso duplicado
Memoria insuficiente
Dirección inválida
Liberación de memoria no asignada
Acceso a página no válida
Archivo de entrada incorrecto
Tamaño de página inválido
Operación desconocida

1.1.12 Requisitos mínimos de implementación

El programa debe incluir:

Clase AVLTree
Clase SplayTree
Clase MemoryManager
Clase Process
Clase Page
Clase Cache
Clase OperationParser

Ejemplo de diseño:

```
class Page {
private:
    int virtualPage;
    int physicalPage;
    string processId;
    bool valid;
    int references;
};

class Process {
private:
    string id;
    int requestedMemory;
```

```

        SplayTree<int, Page> pageTable;
        Cache localCache;
};

class MemoryManager {
private:
    int totalMemory;
    int pageSize;
    AVLTree<int, Page> freePages;
    unordered_map<string, Process> processes;
    Cache globalCache;
};

```

1.1.13 Entregables

Cada estudiante debe entregar:

1. Código fuente en C++.
 2. Archivo README.md.
 3. Archivo de entrada de prueba.
 4. Informe técnico.
 5. Evidencias de ejecución.
 6. Análisis de complejidad.
 7. Explicación del uso de AVL y Splay Trees.
-

1.1.14 Informe técnico

El informe debe incluir:

Introducción
 Descripción del problema
 Diseño de clases
 Estructuras de datos utilizadas
 Justificación del AVL
 Justificación del Splay Tree
 Formato de entrada
 Formato de salida
 Casos de prueba
 Manejo de errores
 Análisis de complejidad
 Conclusiones

1.1.15 Casos de prueba mínimos

Debe probarse:

Creación de un proceso válido
 Creación de proceso con memoria insuficiente
 Finalización de proceso
 Acceso exitoso a página
 Acceso a dirección inválida
 Asignación adicional de memoria
 Liberación parcial de memoria
 Liberación inválida
 Llenado del caché local
 Llenado del caché global
 Reemplazo por LRU
 Múltiples procesos activos

1.1.16 Rúbrica de calificación

Criterio	Puntaje
Diseño general del simulador de memoria	15
Implementación correcta del árbol AVL	15
Implementación correcta del Splay Tree	15
Gestión de procesos y páginas	15
Caché local y caché global	10
Lectura de operaciones por archivo o consola	8
Manejo de errores	7
Métricas y estadísticas	5
Informe técnico	5
Calidad del código, modularidad y estilo	5
Total	100

Detalle por niveles

Nivel	Descripción
Excelente	Implementa correctamente todas las operaciones, usa AVL y Splay Tree de forma funcional y justificada, maneja errores y presenta métricas claras.
Bueno	Implementa la mayoría de operaciones, con pequeños errores en casos límite.
Regular	El simulador funciona parcialmente, pero tiene problemas en asignación, liberación o cachés.
Deficiente	No integra correctamente AVL/Splay Tree o no simula adecuadamente la memoria paginada.

1.1.17 Reglas adicionales

- No se permite usar `std::map`, `std::set` o estructuras equivalentes para reemplazar el AVL o el Splay Tree.
- Sí se permite usar `vector`, `queue`, `stack`, `list` y `unordered_map` como estructuras auxiliares.

1.1.18 Extensiones opcionales

Para obtener puntos extra (hasta 15% extra), se puede incluir:

Visualización del estado de memoria

Exportación de estadísticas a CSV

Simulación de reemplazo de páginas

Pruebas unitarias

1.1.19 Entrega

Un archivo .zip con todos los entregables requeridos y con el nombre de la forma *ApellidoNombre* debe ser subido en el Tec Digital ANTES de la hora de clase del día 27 de mayo.